Irrigation Need

Project Work on Machine Learning and Data Mining M - B2126

Luca Zaffaroni

April 30, 2026

Dataset description

- The Dataset contains features and informations about ground specifics (such as soil properties and composition, climatic conditions etc...) with the goal to predict the need for irrigation,
- The data set for this competition was generated from a deep learning model trained on the Irrigation Prediction dataset. Feature distributions are close to, but not exactly the same, as the original.
- Files : 'train.csv' The competition's training set, used for train validation and test (train size=0.8)

Evaluations

The target is divided into three categories, corresponding to the need for irrigation: low-medium-high
Evaluation is based on **balanced accuracy** between the predicted class and observed target.

Data types

- Qualitative features
  - Nominal(8): (soil type, crop type, crop growth stage, season, irrigation type, water source, region, mulching used)
  - Ordinal (None)
- Quantitative features
  - Interval (1): (Temperature C°)
  - Ratio (10): (soil ph, soil moisture, humidity, organic carbon, electrical conductivity, rainfall mm, sunlight hours, wind speed km-h, field area hectare, previous irrigation mm)
- Target (irrigation need) $\rightarrow$ *Nominal*

Feature distributions

Figure: Features and target distributions: with the exception of soil pH and rainfall (mm), all features show uniform distributions.

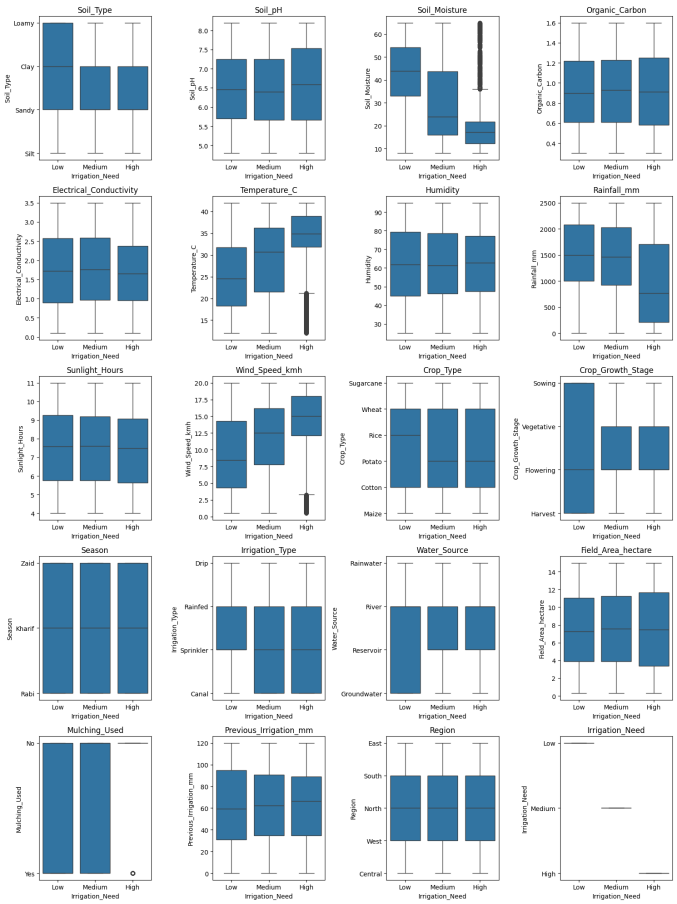

Figure: Boxplots show how the target value "High" is critical, being the only value with distributions outside the whiskers with lots of outliers

More insights...

It's worth noticing the imbalance between the already mentioned target class 'High' with respect to the others classes
Other important informations can be obtained with a dataframe inspection via pandas API such as:
- df.sample(n)
- df.info()
- df.describe()

Our dataset has 630000 entries, 0 to 629999 and 20 columns.
The dtypes are: float64(11), object(9)
Memory usage: 100.9+ MB
Finally, our dataset has no null-values.

Methods

Different possibilities..

- PCA
- SelectKBest
- Other methods (Leave-One-Feature-Out)

...but same results..

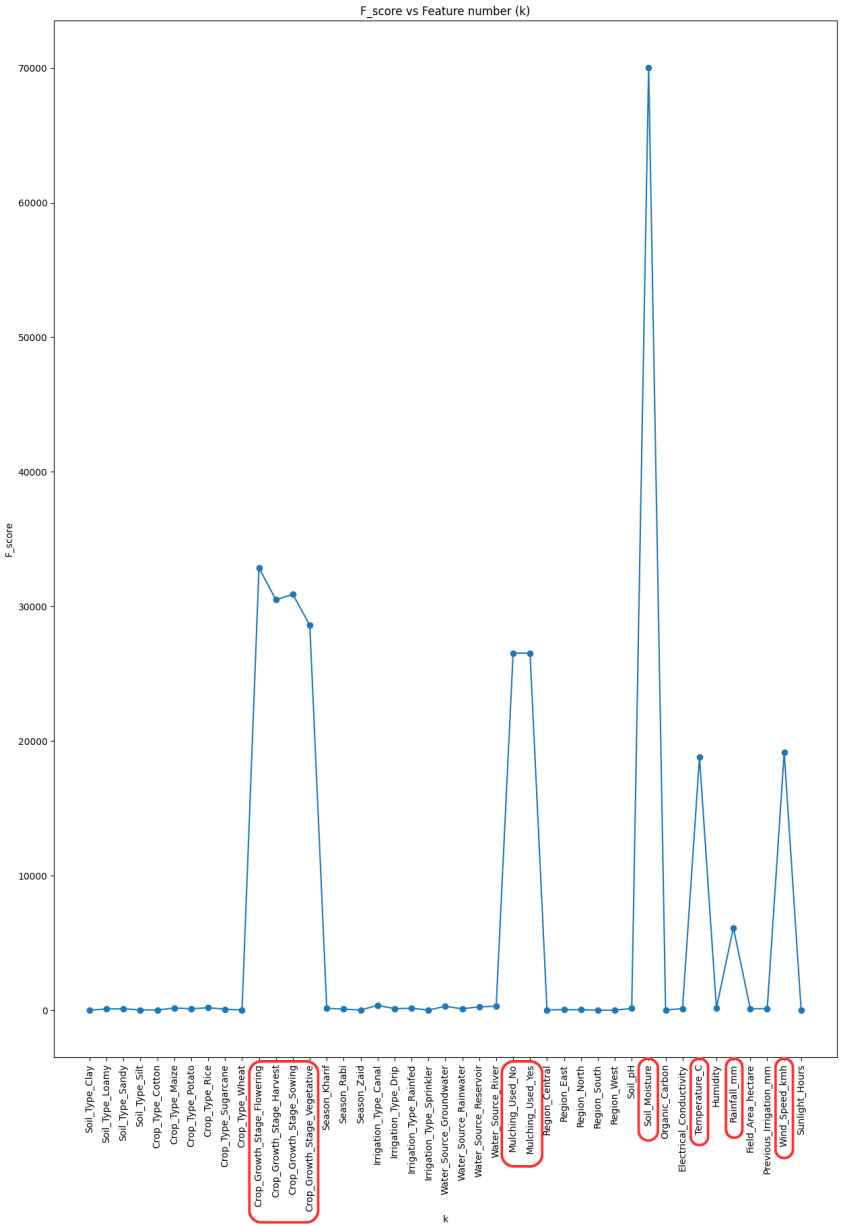

Figure: The six best features are widely more relevant than the others

Workflow and inspected models

The new training set obtained has 85% of rows of the dataset file - 15% for testing - and the six columns divided as:
cols_to_transform = ['Soil_Moisture','Temperature_C','Rainfall_mm','Wind_Speed_kmh']
cols_to_one_hot_encode = ['Crop_Growth_Stage','Mulching_Used']

After first results, both **sklearn.QuantileTransformer** and **sklearn.PowerTransformer** were tested as possible transformations for float values: QuantileTransformer aimed to improve the uniform distribution and give robustness to outliers, but left the results almost unchanged, whereas PowerTransformer aimed to standardize the (really) different ranges of the features, and,improvement of the final score on balanced_accuracy of +0.5% on the train set and +0.3% on the test set, was eventually implemented.
Categorical values were encoded with One-Hot-Encoding method **sklearn.OneHotEncoder**
Finally, the results of such transformations were merged.

As a preliminary examination, the following models were quickly valued based on accuracy:

Model	**Score**	**Params**
Gaussian N.B.	0.772	'var_smoothing': 0.001
Log. Regression	0.871	'C': 1, 'penalty': 'l2'
SGD Classifier	0.865	'alpha': 0.0001, 'class_weight': 'balanced'
Decision Tree	**0.985**	'class_weight': None, 'max_depth': 9
Perceptron	0.835	'alpha': 0.0001, 'penalty': 'l2'
AdaBoost	**0.958**	'learning_rate': 1.25, 'n_estimators': 50
Random Forest	**0.985**	'class_weight': 'balanced', 'max_depth': 17
Linear SVC	0.865	'C': 1, 'class_weight': 'balanced'

Table: Accuracy scores by different models

Accurate examinations

The following examination considers all the models which have given optimal results: Decision tree classifier, Random Forest and AdaBoost.
The final results were obtained after this model selection process and the fine tuning of their hyper-parameters.
For this purpose, **sklearn.GridSearchCV** was used with cross-validation split = 5 and scoring = 'balanced_accuracy'.

Grid Search results

Decision Tree	**0.970**	'class_weight': 'balanced', 'criterion': 'gini', 'max_depth': 9, 'max_features': None, 'min_impurity_decrease': 0.0, 'min_samples$_{l}eaf'$: 2,$'$ *min_samples_split*$'$: 5,$'$ *splitter*$'$:$'$ *best*$'$
AdaBoost	0.712873	'learning_rate': 1.25, 'n_estimators': 50
Random forest	0.966256	'class_weight': 'balanced', 'max_depth': 10, 'max_features': 'sqrt', 'min_samples_leaf': 2, 'min_samples_split': 3, 'n_estimators': 20

Table: Balanced accuracy score for selected models

Results with decision tree classifier

The balanced accuracy on train set is 96.988%

The balanced accuracy on test set is 96.677%

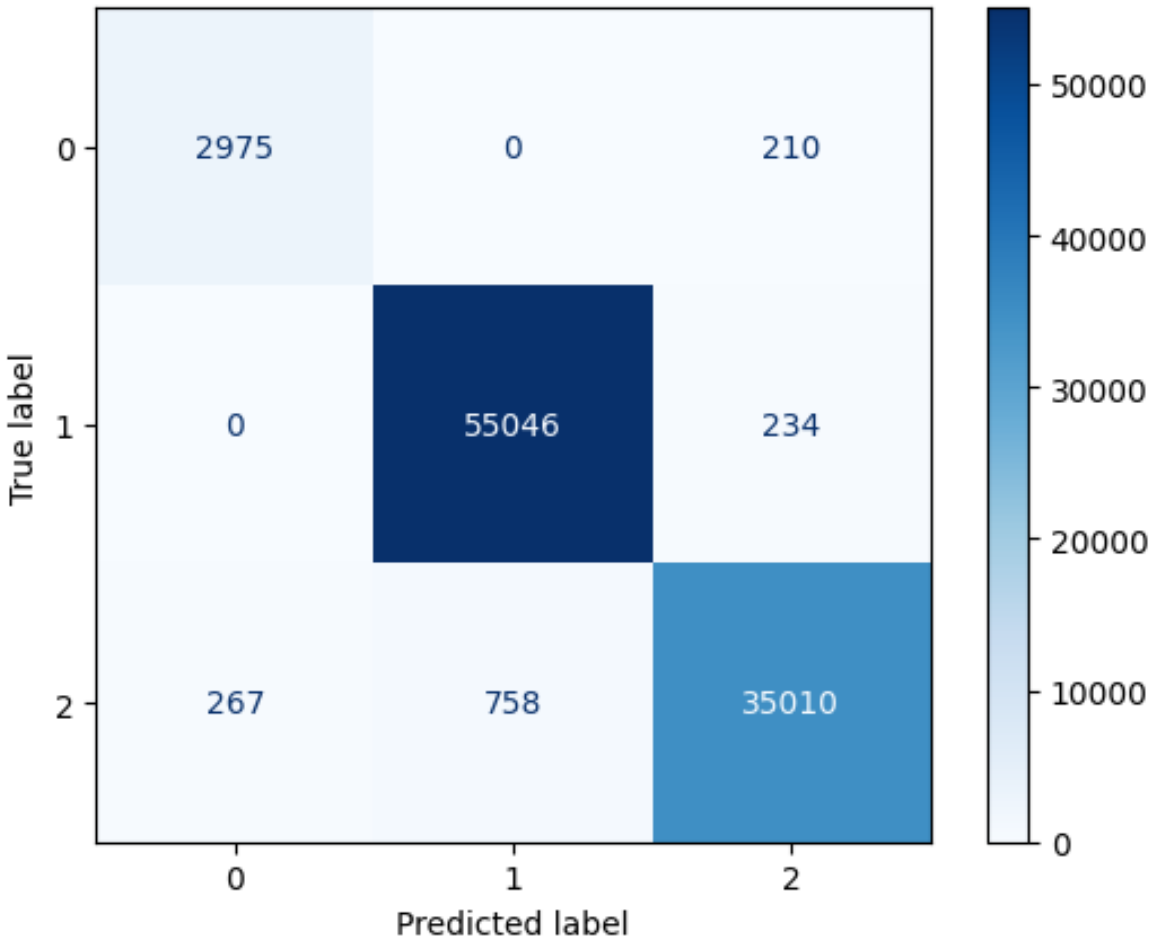

Figure: Label encoder mapping $\rightarrow' High' : 0,' Low' : 1,' Medium' : 2$

Final comments

Given the confusion matrix on the previous slide, the major issues to tackle for better performances could focus on the class 'Medium', giving 100% of the misclassifications (477 with class 'High', 992 with class 'Low'), as well as implementing more regularization techniques (e.g. rounding the numerical features, optimizing other model hyper-parameters).

Final Remarks

Overall, the results were optimal for the task and captures the underlying data patterns correctly.

Thanks for your attention